electronics

MDPI

Article

STE-YOLO: A Surface Defect Detection Algorithm for Steel Strips

Dongming Li [1,2], Erfu Wang [1,2], Zhiyi Li [3], Yingying Yin [1], Lijuan Zhang [2] and Chunxi Zhao [1,4,*]

[1] Institute of Information Technology, Jilin Agricultural University, Changchun 130118, China; ldm0214@163.com (D.L.); darkmoontide@icloud.com (E.W.); yyy@jlau.edu.cn (Y.Y.)
[2] College of Internet of Things Engineering, Wuxi University, Wuxi 214105, China; zhanglijuan@cwxu.edu.cn
[3] College of Instrument Science & Electrical Engineering, Jilin University, Changchun 130012, China; zyli6522@mails.jlu.edu.cn
[4] Information Center, Jilin Agricultural University, Changchun 130118, China
* Correspondence: zcx@jlau.edu.cn

Abstract: To accurately detect defects, we propose an enhanced model based on YOLOv8, named STE-YOLO. To address the aforementioned challenges, this paper adopts YOLOv8 as the improved model. The structure of this paper is as follows: We enhance the model's feature extraction and small detail recognition by integrating GhostConv into partial convolutions. In order to address the attention bias of the model, we introduce a Bottleneck Transformer self-attention convolution layer that effectively improves localization box accuracy. For the problem of defect category mismatches, we exploit the C2f-LSKA attention mechanism in the model head to address this issue. The experimental results indicate that the improved model achieves a mean average precision (mAP) of 79.0%, compared to 65.8% for the original model, marking an improvement of 13.1%. STE-YOLO significantly increases the precision of detecting surface defects on strip steel.

Keywords: surface defect detection; strip steel; YOLOv8; STE-YOLO; GhostConv; Bottleneck Transformer; self-attention convolution; localization box accuracy; C2f-LSKA attention mechanism; model precision improvement

check for updates

Academic Editor: Sara Deilami

Received: 7 December 2024
Revised: 22 December 2024
Accepted: 24 December 2024
Published: 26 December 2024

Citation: Li, D.; Wang, E.; Li, Z.; Yin, Y.; Zhang, L.; Zhao, C. STE-YOLO: A Surface Defect Detection Algorithm for Steel Strips. *Electronics* **2025**, *14*, 54. https://doi.org/10.3390/electronics14010054

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Steel strips are materials used in industrial production and are directly related to product quality and service life. Yet, surface defects frequently diminish their quality. These defects can arise from physical damage during manufacturing, inherent material flaws present from the time the steel is made, and handling errors at subsequent stages of processing.

Surface defects in steel strips prevent them from performing optimally, lower their strength and toughness, and reduce their corrosion resistance. Also, flaws might ruin their cosmetic appearance and negatively affect their marketing. Of greater concern, however, are the potential safety risks posed by undetected defects, especially in critical applications such as construction structures or automotive components.

The causes of such defects include surface overheating, decarburization, relaxation, distortion to high sulfur and phosphorus impurities, inclusions, incomplete removal of iron oxides, and residual liquids from the annealing process. Other factors include foreign matter on the rolled surface, insufficient roll hardness, and scratches caused by sharp objects during handling or transportation. These defects can degrade the properties of the strip and, thus, pose a safety risk. Therefore, modeling the detection of surface defects in steel strips is very important in the industry.

Historically, defect detection has relied heavily on manual inspection and the results depended heavily on the experience and fatigue level of the inspector. With the rise of automation technology, many previous manual tasks were gradually replaced by automated inspection machines [1]. Along with the emergence and rise of artificial intelligence technology [2] there has been simultaneous facilitation in the development of industrial defect detection and a reduced reliance on human workers [3]. There is an urgent and growing need for faster processing speeds in related industries. Organizations require AI algorithms that can meet both speed and accuracy requirements [4].

Northeastern University's Surface Defect Database is a key resource that categorizes six common surface defects in hot rolled steel strips: rolled-in scale (RS), patch (Pa), crazing (Cr), pitted surface (PS), inclusion (In), and scratch (Sc). This database contains 1800 grayscale images, with 300 samples for each defect type. This dataset is crucial for the research and development of surface defect detection algorithms.

Traditional methods use manual features to classify surface defects. These methods mainly include LBP (local binary pattern) [5], HOG (histogram of oriented gradients) [6], GLCM (gray level co-occurrence matrix) [7], and other statistical features [8,9]. Although these methods significantly improve the detection rate of surface defects and reduce error probability, they are very sensitive to external interference factors. Complex calibration preparations are required for defect detection, and if environmental factors in the factory change, thresholds must be adjusted, or the algorithm is likely to fail due to the new environment. While LBP shows promise in texture analysis, its performance in defect detection suffers from its sensitivity to noise, limited capture of global context due to its localized nature, and poor robustness to illumination and rotation changes. HOG, although effective in capturing shape information, faces challenges due to its high computational requirements, sensitivity to shape distortion, potential loss of critical detail information during feature extraction, and dependence on carefully selected parameters. Similarly, GLCM, while capable of providing rich texture features, is hampered by high computational complexity, difficulty in selecting the most effective defect classification features from numerous outputs, sensitivity to noise, and the need for careful parameter tuning.

Deep learning methods have advantages in defect detection that other methods do not. For example, end-to-end detection methods can be used to extract image features, reducing manual work. Deep learning-based object detection algorithms mainly include single-stage and two-stage algorithms. Algorithms such as R-CNN [10], Fast R-CNN [11], Faster R-CNN [12], SSD [13], and the YOLO series [14,15] have improved object detection methods, thereby enhancing defect detection effectiveness. Duo et al. [16] introduced an adaptive focusing feature network (AFF-Net) with a focused feature pyramid network (Foc-FPN), achieving superior detection performance of steel surface defects through adaptive feature weighting and multiscale fusion. Li et al. [17] proposed an end-to-end steel strip surface defect recognition system based on deep convolutional neural networks. In [18,19], Li et al. developed an improved YOLOv4 algorithm for steel strip surface defect detection by adding CBAM and replacing the original path aggregation network.

Dong et al. [20] presented EAIRNet, a network for strip steel defect detection using edge-aware interaction and regional attention, demonstrating its effectiveness on the SD-Saliency-900 dataset compared to the state-of-the-art methods.

YOLO [21] (you only look once) is an object detection algorithm proposed by Joseph Redmon and Ali Farhadi in 2015. Its core idea is to simplify the object detection problem into a single regression task, where the neural network directly predicts the bounding boxes and class information of objects from the input image. Compared to other object detection methods, the biggest advantage of YOLO is its high detection speed while maintaining

high accuracy [22]. Its wide applications include autonomous driving, video surveillance, and other automation fields.

YOLOv8 is a version of the YOLO series launched by Ultralytics. This version has improved in terms of architecture, such as using a more efficient backbone network and an anchor-free detection head with new loss functions that were not used in previous versions. In addition to object detection, the YOLO model can also be used for image classification, instance segmentation, and pose estimation. Compared with YOLOv5, YOLOv8 has significantly improved model accuracy, flexibility, and generalization ability while still maintaining high inference speed. Another improvement in YOLOv8 involves replacing the C3 module with C2f to reduce model weights. Moreover, a task-oriented allocation strategy is undertaken that optimizes the distribution of both positive and negative samples to further enhance detection performance and overall effectiveness.

Computational cost: Current research in the field of industrial inspection underscores an urgent demand for intelligent, cost-efficient processing algorithms. Considerations range from handling big data to achieving real-time performance and minimizing access latency. Questions arise about whether the computational cost is justified by its usage and energy consumption, and the expense of maintaining the service in production. To manage large real-time datasets, substantial computational resources are required, which in turn increases the cost of hardware and algorithm optimization. Moreover, the cost of buying and maintaining HPC resources can eat a lot into one's budget. The hardest thing involves balancing quality with efficiency. Therefore, lightweight models are preferred in industrial inspection, such as MobileNet [23–25], SqueezeNet [26,27], and ShuffleNet [28,29]. These models improve detection efficiency by reducing parameters and computational load. MobileNet achieves efficient computation through depth-wise separable convolutions, suitable for real-time detection. SqueezeNet uses Fire modules to reduce the number of parameters, suitable for resource-constrained devices. ShuffleNet achieves efficient detection in environments with extremely low computational requirements through channel shuffling and group convolutions. Although MobileNet may not be precise enough for complex tasks, SqueezeNet performs poorly in highly complex tasks and has limited feature extraction capabilities, and ShuffleNet, while efficient, may have some shortcomings in high-precision requirements and complex feature extraction. GhostConv [30,31] reduces computational and parameter requirements by generating "ghost features", significantly lowering memory and computational overhead while maintaining high performance. This method is particularly suitable for lightweight neural networks, enabling the model to run efficiently while maintaining good detection accuracy. It is very suitable for embedded devices and real-time industrial inspection, especially in environments with limited computational resources and energy consumption. Therefore, we decided to improve our model based on GhostConv.

We propose an enhanced model based on YOLOv8 for detecting surface defects in steel strips. The main contributions of this paper are as follows:

1. By modifying the standard convolutional layers in the model backbone to GhostConv, the computational cost and memory usage of YOLO are effectively reduced, alleviating the computational resource demands of other improvements. This improvement reduces performance requirements under resource-constrained conditions while improving computational accuracy, achieving lower latency.
2. Integrating the Bottleneck Transformer into the connection between the model backbone and neck, combining the global context modeling capability of the Transformer with YOLO's advantages in real-time object detection. This integration takes charge of complex neighborhoods in the descriptor space, leading to a fine-tuned semantic deskew that results in increased positive recognition.

3. The integration of the LASK attention algorithm in the model head considerably increases accuracy in industrial inspection. LASK enables the model to focus on the most critical detection areas, even within complex, highly interfering backgrounds—this is necessary for detecting defects extremely similar to the background and helps YOLO in detecting even small defects in more complicated scenes, therefore improving the overall accuracy and efficiency of detection.

2. Related Works

The emergence of computer vision in recent years has tremendously aided in the inspection of metal surfaces, significantly improving the situation. However, progress in metal surface defect detection still faces many challenges, such as balancing detection accuracy with processing speed, the similarity between defects and their backgrounds, and the accuracy in detecting small targets.

2.1. Traditional Machine Learning Methods

Traditional image processing algorithms often struggle with the challenges posed by small defect features and low contrast between defects and their backgrounds. Jiawei Zhang et al. [32] (2020) proposed a defect detection method based on fuzzy metrics to improve the detection accuracy of different defect types. This method first analyzes the statistical information of defect-free images, assuming that the background intensity follows a Gaussian distribution. Then, by testing the histogram of the image and estimating the parameters of the background model, a membership function is defined. This method can evaluate the extent to which each grayscale value belongs to a defect and assign a threshold to each pixel. Yue et al. [33] (2021) proposed an improved bat algorithm optimization method for classifying metal surface defects via a BP neural network. This study first used the local binary pattern (LBP) algorithm to extract features from six types of defect images and created a feature sample library. Then, by using the WG-BA-BP network with weighted experience factors, different features were classified for defect images. Ashour et al. [34] (2018) proposed a method combining discrete shearlet transform (DST) and the gray level co-occurrence matrix (GLCM) to classify the surface defects of steel strips. The main goal of this study was to improve the accuracy of defect classification in the presence of high similarity and variability between defect types.

2.2. Deep Learning Methods

Guan et al. [35] (2020) proposed a detection algorithm based on an improved deep learning network model. Their method uses a pre-trained VGG19 model to extract feature images at different levels, improving the classification accuracy of steel surface defects. This method introduces a new VSD network, which performs better in classification accuracy and convergence speed than existing methods such as ResNet and VGG19. Deshpande et al. [36] (2019) proposed a single-shot convolutional neural network for steel surface defect detection. This method aims to address the issue of high training data requirements in practical applications, which can often become a major problem for model training. Their method reduces data requirements while achieving real-time defect detection with high-quality control. Y.Z. et al. [37] (2024) introduced a YOLO model for steel infrared images, integrating a coordinate attention mechanism within the C2F framework, and improving feature fusion with BiFPN. Guo et al. [38] (2022) proposed the MSFT-YOLO model, an improved version of YOLOv5, integrating Transformer modules and BiFPN to address the two main issues of feature extraction and multi-scale feature fusion. Xia et al. [39] proposed an improved YOLOv5s model to reduce the low detection efficiency and accuracy caused by background texture features interfering with the target. Their method introduces a new

reparameterized large kernel C3 module and a multi-path spatial pyramid pooling feature fusion structure, enhancing the model's feature extraction capability and adaptability to scale changes under complex texture interference. Li et al. [40] proposed a hybrid architecture combining convolutional neural networks and Transformer encoders to mitigate the limitations of CNNs in capturing global features. Their proposed CNN-T network demonstrated superior performance compared to traditional CNN models such as GoogLeNet, MobileNet v2, and ResNet-18, as well as pure Transformer networks. The CNN-T model effectively integrates the strong inductive biases of CNNs with the global modeling capabilities of Transformers, achieving higher classification accuracy with fewer parameters and floating-point operations. Wang et al. [41] (2023) optimized YOLOv8 in their research and proposed UAV-YOLOv8, a small object detection model based on drone aerial scene data. The model introduces Wise-IoU v3 as the bounding box regression loss function and enhances localization capability by employing a wise gradient allocation strategy, which directs the model to focus more on samples of common quality. Lou et al. [42] (2023) proposed DC-YOLOv8, a detection algorithm for small-sized objects, aiming to improve the detection accuracy of camera sensors in complex scenes. This algorithm significantly enhances detection performance through a novel downsampling method, an improved feature fusion network, and a new network structure. On authoritative datasets such as VisDrone, TinyPerson, and PASCAL VOC 2007, DC-YOLOv8 outperforms YOLOv8s in terms of mAP, precision, and recall, particularly excelling in small object detection.

3. Methods

YOLOv8 [43] is the most recent algorithm in the YOLO series, designed for tasks such as object detection, image classification, and instance segmentation. In this study, we focus on keypoint detection and use the smaller YOLOv8n variant as our base model. Figure 1 illustrates the overall architecture diagram of YOLOv8, including the input layer, the backbone network, the neck network, and the head network. Among them, the backbone network is responsible for extracting features from the input image, the neck network fuses features from different layers, and the head network is responsible for the final classification and localization of defects. The figure provides an intuitive view to help readers understand the basic structure of YOLOv8. The input portion of the image is a sample input from the dataset we used for our experiments.

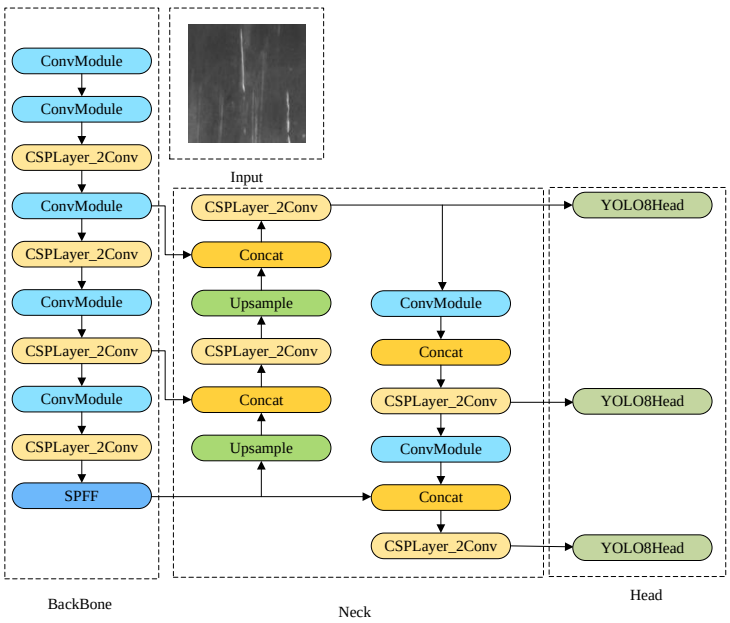


Figure 1. YOLOv8 overall architecture diagram.

Figure 2 illustrates the overall architecture of our proposed STE-YOLO model. Compared to the original YOLOv8 architecture in Figure 1, the STE-YOLO model integrates the GhostConv module in the backbone network, introduces the Bottleneck Transformer module between the backbone and neck networks, and uses the C2f_LSKA attention mechanism in the head network. These improvements aim to improve the feature extraction capability of the model, the global information capture capability, and attention to subtle defects.

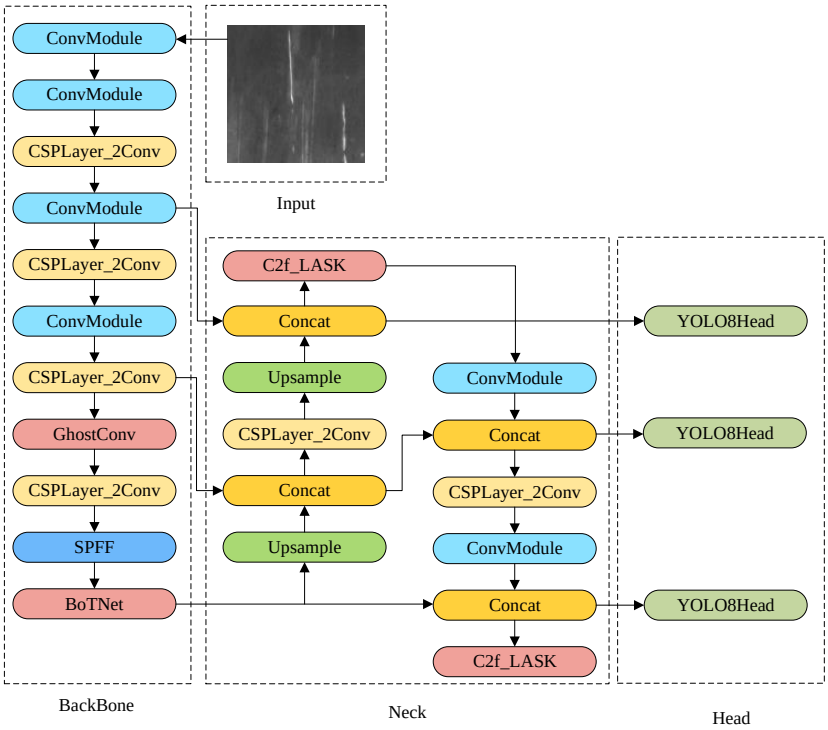


Figure 2. STE-YOLO overall architecture diagram. The overall architecture of the proposed SET-YOLO framework is as follows. The model takes the strip defect data as input and extracts the features of the defects using our designed backbone network, using GhostConv and Bottleneck Transformer models for feature extraction. Then, the model attention is improved by our use of improved C2f, i.e., C2f_LASK. Finally, the bounding box is predicted using the detection head module.

3.1. Ghost Conv

In our research, we introduce Ghost convolution (Ghost Conv) into the backbone network of the model to replace ordinary convolution. This technique aims to generate more features with fewer parameters. Traditional convolutional layers in deep neural networks consist of two main parts: standard convolution and linear operations. The Ghost Conv module reduces the number of parameters and computational complexity compared to traditional convolution while maintaining the original output feature map size.

Ghost Conv includes three steps: applying standard convolution, generating Ghost features, and merging the generated feature maps. Figure 3 details the structure of the GhostConv module. The module first generates a small number of eigenfeatures using standard convolutional operations and then generates a large number of "ghost features" using inexpensive linear transformations. The GhostConv module is designed to reduce the number of parameters and computational effort of the model while maintaining feature richness.

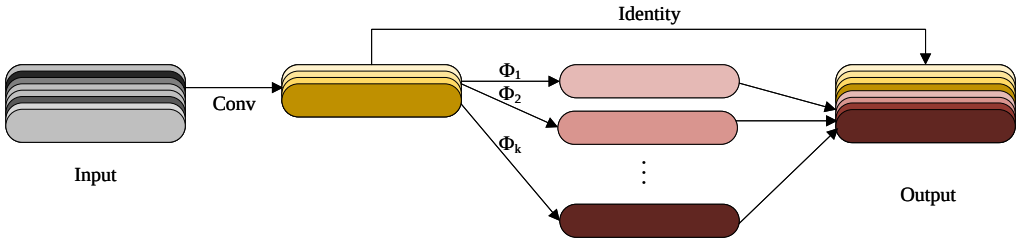


Figure 3. The Ghost module.

First, use conventional convolution to obtain the intrinsic feature map.

$$Y' = X * f' \tag{1}$$

Y' is the output of the convolutional layer, typically consisting of a set of feature maps. X represents the input data, such as images or other types of data. f' is the convolutional filter or weight matrix, used to perform convolution operations on the input data.

Next, the operation $\Phi_{i,j}$ is applied to each feature map y'_i in Y' to generate the Ghost feature map y_{ij}.

$$y_{ij} = \Phi_{i,j}(y'_i), \forall i = 1, \ldots, m, j = 1, \ldots, s \tag{2}$$

In the above formula, y'_i represents the i-th intrinsic feature map in Y. y_{ij} represents the j-th feature map generated in the Ghost module. Here, m is the number of feature maps or channels. In the Ghost Conv neural network, each convolutional layer generates a set of feature maps, with each feature map corresponding to a specific feature or pattern. The variable j indicates the position or index within a given feature map. For each feature map, different positions can be determined by j.

Finally, the intrinsic feature map from the first step is concatenated with the feature map obtained from the second step via an identity connection to produce the final output.

The Ghost Convolution module operates by splitting the original convolution layer into two components. First, a small number of convolution kernels generate multiple intrinsic feature maps. Then, inexpensive transformation operations are applied to generate Ghost feature maps. This strategy reduces both the number of parameters and computational complexity. The design philosophy of Ghost Conv leverages feature maps to minimize resource usage, decrease computational costs, and enhance model efficiency. Specifically, let us consider a standard convolutional layer with input channels C_{in}, output channels C_{out}, and a kernel size of $K \times K$. The number of floating-point operations (FLOPs) required for this layer is approximately as follows:

$$\text{FLOPs}_{\text{std}} \approx C_{in} \times C_{out} \times K \times K \times H \times W \tag{3}$$

where H and W are the height and width of the output feature map, respectively.

In contrast, for a GhostConv layer, let us denote the number of intrinsic feature maps as m, the expansion ratio as s, and the kernel size of the linear transformation as 1×1. The number of FLOPs for the GhostConv layer can be approximated as follows:

$$\text{FLOPs}_{\text{ghost}} \approx \frac{1}{s} C_{in} \times C_{out} \times K \times K \times H \times W + \frac{s-1}{s} C_{in} \times C_{out} \times 1 \times 1 \times H \times W \tag{4}$$

If we assume the kernel size of the second transformation 1×1 is small and the value of s is large, such that the number of intrinsic feature maps, m, is small and we are close to the condition of $\frac{1}{s}$ in the expression. Then, the FLOPs can be approximated as follows:

$$\text{FLOPs}_{\text{ghost}} \approx \frac{1}{s} C_{in} \times C_{out} \times K \times K \times H \times W \tag{5}$$

Thus, if we assume $\frac{s-1}{s}$ is close to 1, we can see a reduction in computation by factor s. As an example, consider a convolutional layer, where $C_{in} = 256$, $C_{out} = 512$, and $K = 3$. If we set $s = 2$, the GhostConv module will reduce computational costs by approximately half compared to the standard convolutional layer while retaining the same output feature map size. Furthermore, with larger expansion ratios, the computational saving will be more significant. This shows that GhostConv greatly reduces the computational cost while maintaining feature map size, enabling faster and more efficient computation.

3.2. Bottleneck Transformer

Common surface defects in steel strips pose a significant challenge due to their extensive coverage areas and indistinct features, which result in limited information in the original images. As convolutional neural networks downsample the data through successive layers, the size of the input feature map decreases, leading to a substantial loss of detail, particularly in small target defects. To address this issue effectively, a multi-scale strategy is widely employed to detect small targets. Shallow networks focus on capturing fine-grained details, which helps in extracting features from small targets.

We propose enhancing cross-layer feature connections by incorporating BoTNet at the end of the model's backbone network. This approach facilitates the fusion of shallow local information with deep semantic information, effectively countering the degradation of information caused by the increasing depth of the network.

BoTNet's module structure and design philosophy integrate self-attention mechanisms into the bottleneck blocks of ResNet to achieve improved performance. Specifically, BoTNet replaces the spatial convolutions in the last three bottleneck blocks of ResNet with global self-attention layers. This design reduces the number of parameters while maintaining a comparable computational load, thereby improving performance in instance segmentation and object detection tasks.

The basic structure of BoTNet is a hybrid model that combines both convolutional and self-attention mechanisms. Essentially, BoTNet replaces the 3×3 convolutions used in bottlenecks, such as those in ResNet50, with multi-head self-attention, as illustrated in Figure 4.

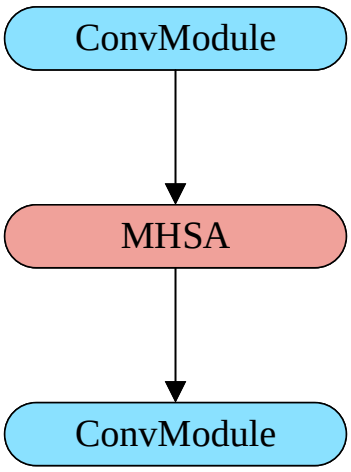


Figure 4. Bottleneck transformer.

To improve the strip steel defect detection model, we introduce a model based on the self-attention mechanism. This model replaces certain convolutional layers in ResNet with multi-head self-attention layers, enabling the modeling of long-range dependencies. The key feature of the BoT model is its use of global self-attention to process and aggregate

the information captured by convolutional feature maps. The multi-head self-attention structure is in Figure 5.

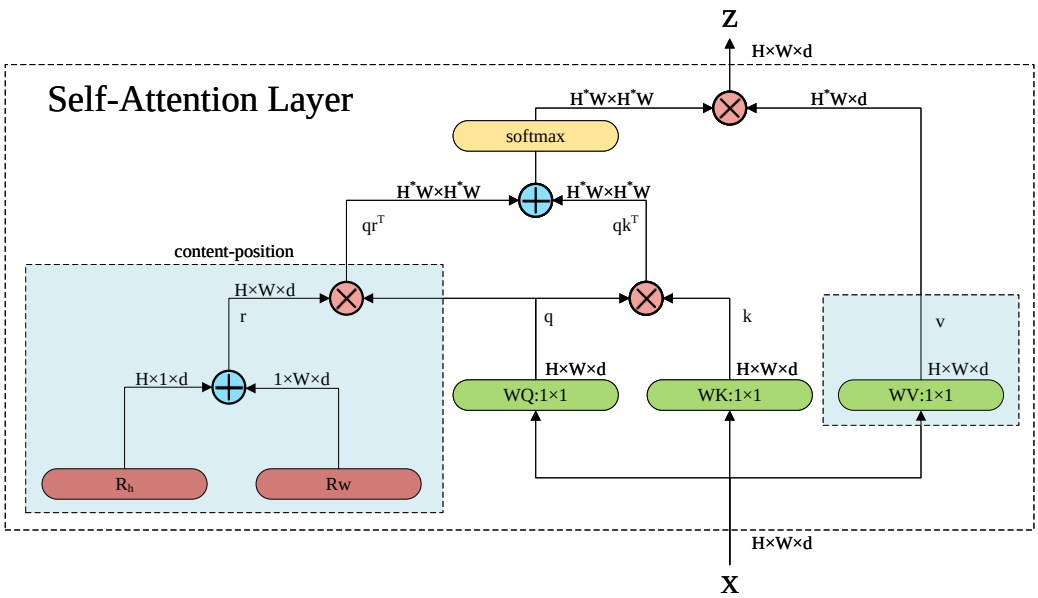


Figure 5. Multi-head self-attention.

$$[MHSA(x) = \text{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)v] \tag{6}$$

The input size of multi-head self-attention is $H \times W \times d$, where H and W represent the height and width of the input feature matrix, and d denotes the dimension of a single token. The number of tokens is $H \times W$.

First, two learnable parameter vectors, R_h and R_w, are initialized to represent the positional encodings for different heights and widths, respectively. These are then combined using broadcasting, so the encoding at position (i, j) is the sum of the two d-dimensional vectors, R_{h_i} and R_{w_j}. This simplifies the $H \times W \times d$ encodings to $(H + W) \times d$. Additionally, the positional encoding is incorporated into the attention mechanism, obtained by performing matrix multiplication with the query matrix. The result is combined with the query and key matrices, and the final attention is computed using the softmax function. Here, $\oplus$ and $\otimes$ denote element-wise addition and matrix multiplication, respectively.

In the above formula, q, k, and v represent the query, key, and value matrices, respectively, with d_k indicating the dimensionality of the key. This design allows the Bottleneck model to learn rich associative features across long sequences, resulting in substantial improvements in the task at hand. This model is not only simple and efficient but also has minimal computational overhead during training and inference, thereby minimizing the impact on the performance of the steel strip defect detection model.

3.3. Large Separable Kernel Attention

LSKA (large separable kernel attention) is an attention mechanism designed to address the common issues of computational and memory overhead in visual attention networks. It first decomposes the 2D convolution kernel into sequential horizontal and vertical 1D convolution kernels. In the attention module, large kernel depthwise separable convolution layers are used directly without additional modules. This decomposition method significantly reduces computational complexity and memory usage while maintaining performance similar to standard LKA.The large separable kernel attention structure is in Figure 6.

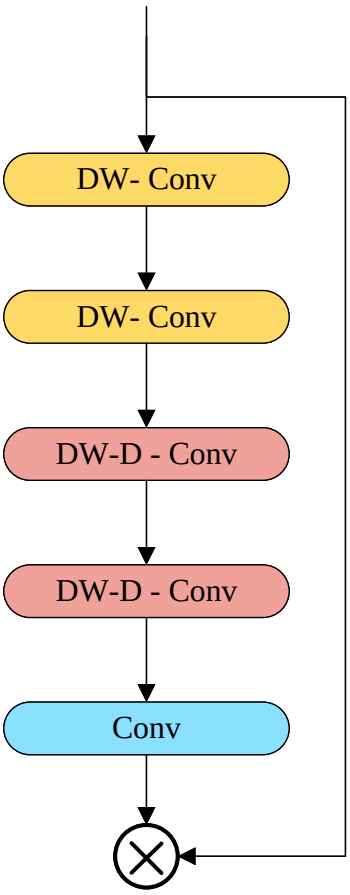


Figure 6. Large separable kernel attention.

The LSKA module efficiently utilizes large kernels by decomposing 2D convolution kernels into sequential horizontal and vertical 1D convolution kernels. This strategy prevents the quadratic growth in the number of parameters that would occur with increased kernel size in LKA-trivial and LKA within VAN.

The LSKA module delivers strong performance across various vision-based tasks, such as image classification, object detection, and semantic segmentation, within VAN. It leverages large kernels while maintaining inference time costs that are comparable to those of smaller kernels used in the original LKA.

The design of the LSKA module introduces a bias in VAN toward the shape of objects, rather than their texture, as the kernel size increases. Additionally, the features learned by the LSKA module emphasize shape information over texture. Compared to previous large-kernel CNNs, the LSKA encodes more shape-related information in feature representations, which helps explain why LSKA-VAN performs robustly.

By splitting the 2D kernel weights of depth-wise convolution and depth-wise dilation convolution into two sequential 1D separable weights, the LSKA module effectively improves upon the LKA structure. We refer to this modified configuration as LSKA, as illustrated in Figure 2. The output of the LSKA module is as follows:

$$\bar{Z}^C = \sum_{H,W} W^C_{(2d-1)\times 1} * \left(\sum_{H,W} W^C_{1\times(2d-1)} * F^C \right) \tag{7}$$

$$Z^C = \sum_{H,W} W^C_{\lfloor \frac{k}{d} \rfloor \times 1} * \left(\sum_{H,W} W^C_{1\times \lfloor \frac{k}{d} \rfloor} * \bar{Z}^C \right) \tag{8}$$

$$A^C = W_{1\times 1} * Z^C \tag{9}$$

$$\bar{F}^C = A^C \otimes F^C \tag{10}$$

The LSKA is essentially a two-layer convolution operation. The first layer processes the input data $\bar{Z}^C$ using a $1 \times \left\lfloor \frac{k}{d} \right\rfloor$ convolution kernel to extract preliminary features. The second layer uses a $\left\lfloor \frac{k}{d} \right\rfloor \times 1$ convolution kernel to further weight and aggregate these features, ultimately producing the output Z^C.

In this context, * and $\otimes$ represent convolution and Hadamard product, respectively. The following are the definitions of the symbols used:

- Z^C: Represents the output of channel C.
- $\sum_{H,W}$: Denotes the summation operation over height H and width W, typically used in convolution operations.
- $W^C_{\lfloor \frac{k}{d} \rfloor \times 1}$: Refers to a weight matrix of size $\left\lfloor \frac{k}{d} \right\rfloor \times 1$, where $\lfloor \cdot \rfloor$ denotes the floor operation. This weight matrix is applied to channel C.
- $W^C_{1\times \lfloor \frac{k}{d} \rfloor}$: Represents another weight matrix of size $1 \times \left\lfloor \frac{k}{d} \right\rfloor$, which is also applied to channel C.
- $\bar{Z}^C$: Represents the input data of channel C.

The formula can be divided into two parts for explanation:

1. Inner summation and multiplication:

$$\sum_{H,W} W^C_{1\times \lfloor \frac{k}{d} \rfloor} * \bar{Z}^C$$

This part represents a convolution operation on the input data $\bar{Z}^C$ using a weight matrix $W^C_{1\times \lfloor \frac{k}{d} \rfloor}$, followed by a summation over the height H and width W. This typically represents a filtering or feature extraction on the input data.

2. Outer summation and multiplication:

$$Z^C = \sum_{H,W} W^C_{\lfloor \frac{k}{d} \rfloor \times 1} * \left(\sum_{H,W} W^C_{1\times \lfloor \frac{k}{d} \rfloor} * \bar{Z}^C \right)$$

The outer layer conducts an additional convolution using the weight matrix $W^C_{\lfloor \frac{k}{d} \rfloor \times 1}$, summing over the height H and width W. This step further weights and aggregates the features from the inner layer to generate the final output Z^C.

The design of these multi-layer convolutions enables more efficient extraction and integration of different features, thereby enhancing the model's capacity for representation.

4. Result

4.1. Experimental Setup

The experiments were conducted on a machine with the configuration shown in Table 1. The software environment of the experimental equipment is shown in the Table 2. Key parameters such as the learning rate and optimizer were chosen based on their impact on model performance, as detailed below.

Table 1. Experimental Hardware Configuration Table.

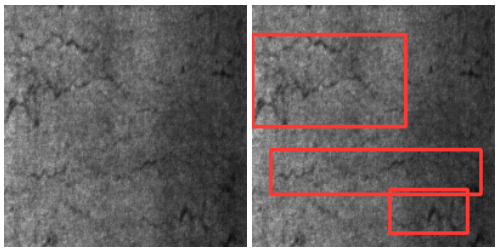
Component	Description	Details
Operating System	Windows 10	Professional Workstation
Processor	Intel (R) Core (TM) i5-9400F	2.40 GHz
Memory	Kingston	32 GB 2666 MHz
Graphics Card	NVIDIA GeForce GTX1080Ti	11 GB VRAM

Table 2. Experimental cla software configuration table.

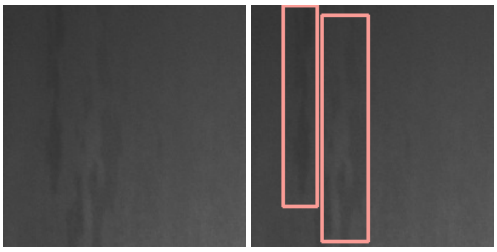
Category	Item
Language	Python 3.9
Platform	PyTorch 2.2
Libraries	Ultralytics 8.3.10
Driver	NVIDIA GeForce Experience 552.22
CUDA	NVIDIA 12.4
CUDNN	NVIDIA 8.9.6

4.2. Experimental Data

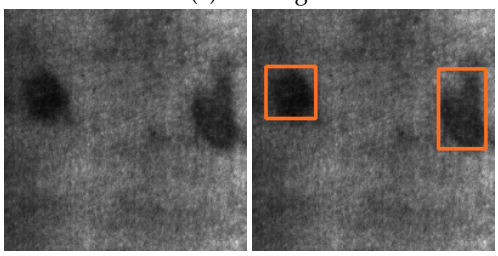
In this study, we utilized the Northeastern University (NEU) surface defect database as our research subject. [44] This dataset contains six typical surface defects of hot-rolled steel strips, which are rolled-in scale (RS), patch (Pa), crazing (Cr), pitted surface (PS), inclusion (In), and scratch (Sc). The dataset consists of a total of 1800 grayscale images, with 300 sample images for each defect type. However, each image may contain multiple sample labels, with a resolution of 200 × 200 pixels. As illustrated in Figure 7, we can observe significant visual differences within defects of the same class, whereas inter-class defects exhibit similar characteristics. Moreover, due to variations in lighting and materials, there are differences in the grayscale of intra-class defect images. Notably, this dataset provides detailed annotations for defect detection tasks, including defect categories and location information, with defect labels and distribution shown in Figure 8. By referencing the NEU surface defect database, we can overcome the challenges of intra-class defect appearance differences and inter-class defect similarities, thus providing valuable data support for the research.



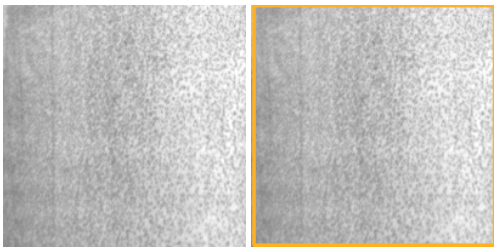
(**a**) Crazing



(**b**) Inclusion

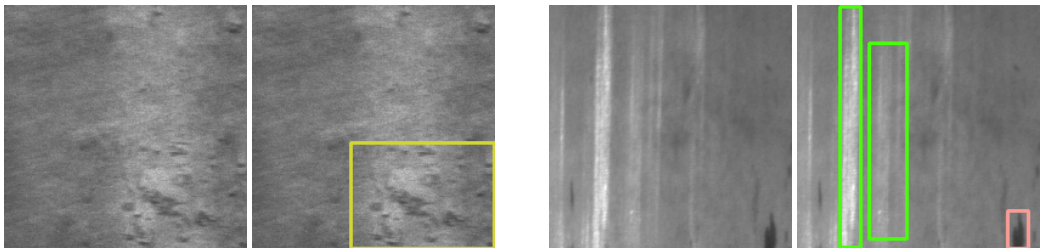


(**c**) Patches



(**d**) Pitted surface

Figure 7. *Cont.*



(**e**) Rolled-in scale (**f**) Scratches

Figure 7. The image shows six pairs of gray scale images. Each pair is labeled with a defect type. The left image of each pair shows a section of the material, while the right image is the same section with one or more pink rectangles overlaid on it. These rectangles mark the location of the described defect on that image.In this table, different colors are used to categorize various types of surface defects; Crazing is highlighted in red, inclusions are in pink, patches are in orange, pitted surface is in beige, rolled-in scale is in yellow, and scratches are in green.

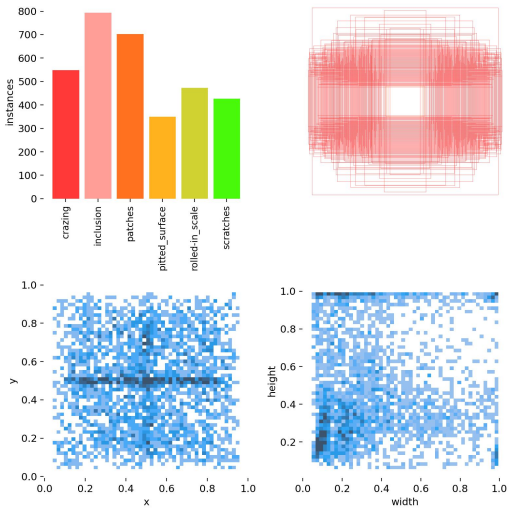


Figure 8. Distribution of defects in the NEU-DET dataset.

4.3. Performance Evaluation

The evaluation metrics for target detection typically include recall, precision, average precision (AP), and mean average precision (mAP), which are used to assess detection efficiency and accuracy. We will employ these metrics to evaluate our results in the upcoming trial. First, we clarify some key concepts: A true positive (TP) indicates the correct identification of a positive case with the accurate defect area detected. A false positive (FP) occurs when a positive prediction is made, but the defect area identified is incorrect. A true negative (TN) means a negative sample is correctly identified by not detecting any defect area. A false negative (FN) indicates a negative recognition when the correct defect area is not identified despite its presence.

Precision is the accuracy of detecting objects, indicating how many detections are correct. The calculation formula is as follows:

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{11}$$

where P represents precision, P represents true positive, and FP represents false positive.

Recall is the model's ability to identify all object instances in an image. The calculation formula is as follows:

$$\mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{12}$$

where R represents recall, TP represents true positive, and FN represents false negative.

AP (average precision) is determined by the area under the precision–recall curve, where $(r_{i+1} - r_i)$ represents the recall rate, and $P(r_{i+1})$ represents precision.

$$\mathrm{AP} = \sum_{i=1}^{n-1} [r_{i+1} - r_i][P(r_{i+1})]. \tag{13}$$

mAP (mean average precision) represents the average AP of all categories in the dataset. The calculation formula considers the number of example categories m in the test set. Specifically, mAP@0.5:0.95 refers to the average mAP over an IoU threshold range from 0.5 to 0.95, with a step size of 0.05. In contrast, mAP@0.5 refers to the average precision calculated at an IoU threshold of 0.50, which is a measure of the model's accuracy considering only "easy" detections.

$$\mathrm{mAP} = \frac{\sum_{i=1}^{K} \mathrm{AP}_i}{K} \tag{14}$$

The F1 score is a harmonic mean of two other critical metrics: precision and recall. The formula is as follows:

$$\mathrm{F}_1 = 2 \cdot \frac{\mathrm{P} \cdot \mathrm{R}}{\mathrm{P} + \mathrm{R}} \tag{15}$$

FPS (frames per second) is a performance metric indicating the number of frames the model processes per second.

$$\mathrm{FPS} = \frac{1}{\text{Processing time per frame}} \tag{16}$$

4.4. Hyperparameters

Table 3 shows the hyperparameters used in our experiments.

To optimize the training performance of the model, we made the hyperparameter settings as shown in Table 3: the number of training epochs is set to 200, and the optimizer chosen is AdamW, which combines the advantages of momentum and RMSprop, and incorporates weight decay. This optimizer offers benefits in terms of convergence speed and is easier to tune. The number of workers is set to 4, considering that the local device has 6 cores and 6 threads, and some resources need to be reserved to support remote monitoring. We adopted a cosine learning rate scheduling strategy, with an initial learning rate (lr0) of 0.004 and a final learning rate coefficient (lrf) of 0.0005. Considering that the dataset may have a certain degree of noise and background interference, which can lead to model overfitting, we chose a smaller final learning rate and used the cosine annealing strategy to allow the model to gradually converge to a better local minimum, improving the model's stability. Additionally, we enabled the memory cache mechanism (Cache=True) to speed up data loading and significantly shorten training time. Our experiments found that this set of parameters can ensure the model's convergence speed while achieving higher validation accuracy.

Table 3. Experimental hyperparameters in Table.

Parameter	Value
epochs	200
optimizer	AdamW
workers	4
cosine learning rate	True
lr0	0.004
lrf	0.0005
cache	True

4.5. Experiment Result

The Table 4 illustrates the detection performance of the STE-YOLO model on the NEU-DET dataset. It includes precision, recall, mAP50, and mAP50-95 data for the entire dataset and its six subclasses. The comparison of these data indicates that our proposed method can accurately identify most defects. Our approach performs well across various detection aspects, particularly for patches, achieving a mAP of 94.9%, and for scratches, with a mAP of 91.9%. However, the performance for crazing, with a mAP@0.5:0.95 of only 44.6%, is relatively poor. This is likely due to the tendency of crazing patterns to blend in with the background, making defect recognition challenging.

Table 4. Illustrates the performance of the STE-YOLO model for each type of defect in steel strips. The table presents metrics such as precision, recall, mAP50, and mAP50-95 for different configurations.

Category	Precision (%)	Recall (%)	mAP50 (%)	mAP0.5:0.95 (%)
All	0.728	0.745	0.790	0.500
Crazing	0.653	0.346	0.485	0.213
Inclusion	0.758	0.843	0.882	0.547
Patches	0.852	0.927	0.962	0.708
Pitted surface	0.796	0.750	0.800	0.534
Rolled-in scale	0.623	0.642	0.692	0.415
Scratches	0.685	0.962	0.921	0.584

4.6. Ablation Experiment

In our analytical study, we designed a detailed algorithm based on YOLOv8 as a benchmark model. To reasonably assess the application value of the proposed improvements for strip steel defect detection, we conducted joint analysis experiments using the NEU-DET dataset for testing. The results are listed in Table 5 where YOLOv8 represents the original YOLOv8 model, YOLOv8-GhostConv represents the model in which the eighth layer convolution in YOLOv8's neck is replaced with the GhostConv module, YOLOv8-BoTNet represents the model with only the BoTNet structure added to the original structure in YOLOv8's neck, YOLOv8-C2f_LSKA represents the model with LSKA attention mechanisms added to the first and third pooling layers in the head, and STE-YOLO represents the algorithm we proposed.

Table 5. Presents the results of the ablation study, showing the performance of the SET-YOLO model for different configurations of defect detection in steel strips. The metrics include precision, recall, mAP50, mAP50-95, F1, and FPS.

YOLOv8	GhostConv	BoTNet	C2f_LSKA	Precision (%)	Recall (%)	mAP50 (%)	mAP50-95 (%)	F1	FPS
✓				59.8	70.7	65.8	37.8	0.75	70.31
✓	✓			60.7	71.0	71.7	41.4	0.62	65.67
✓		✓		62.2	68.2	69.0	38.9	0.63	68.40
✓			✓	65.4	67.3	69.6	38.5	0.60	60.66
✓	✓	✓		71.9	73.3	77.6	44.4	0.72	65.2
✓		✓	✓	70.7	71.9	76.6	43.1	0.70	62.1
✓	✓		✓	70.6	74.8	78.1	43.8	0.72	66.0
✓	✓	✓	✓	72.8	74.5	79.0	50.0	0.72	61.40

In our ablation study, we adopt a sequential integration approach for evaluating the contributions of each module. First, we replace standard convolutional layers in the YOLOv8's backbone with GhostConv modules. This step prioritizes reducing the computational overhead and model size while maintaining reasonable feature representation capabilities. GhostConv addresses the need for resource-efficient operation, which helps create a lean and effective backbone for the subsequent modules to build upon.

Next, we incorporate the Bottleneck Transformer (BoTNet) module into the connection between the backbone and the neck of the network. This aims to capture long-range dependencies in the feature maps. Bottleneck Transformer provides a mechanism for capturing and processing global context within the feature maps, enabling the network to understand global features in the image, as opposed to simple local connections.

Finally, we introduce the large separable kernel attention (LSKA) mechanism to the first and third pooling layers in the head of the network. This step is to focus the attention on the most salient defect-related areas, which reduces noise interference and increases the localization accuracy of the defect regions. This stage refines the network's focus on salient regions related to defects and improves the overall detection accuracy.

This sequential integration is based on building the model from its foundation—focusing on efficient computation first, before adding mechanisms for long-range dependencies, and finally, fine-tuning with attention-based refinement. This ensures a balanced and efficient improvement of the model.

4.7. Comparative Experiments

To thoroughly evaluate the detection performance of the proposed STE-YOLO model, this study compares it with several state-of-the-art object detection models. These models include Faster R-CNN, YOLOv3, YOLOv5, YOLOv6, RT-DETR-ResNet50, and ViTDet. The corresponding experimental results are presented in the following Table 6.

Table 6. Presents the comparative experimental results, showing the performance of the SET-YOLO model for different configurations in defect detection in steel strips. The metrics include precision, recall, mAP50, mAP50-95, F1, and FPS.

Model	Precision (%)	Recall (%)	mAP50 (%)	mAP50-95 (%)	F1	FPS
YOLOv8	59.8	70.7	65.8	37.8	0.75	70.31
YOLOv6	49.0	69.2	60.9	27.3	0.54	80.30
YOLOv5	59.3	72.0	68.3	38.9	0.61	68.55
YOLOv3	49.0	69.2	56.2	27.3	0.52	111.58
Fast R-CNN	-	-	56.5	-	-	-
RT-DETR-ResNet50	54.2	47.7	49.8	26.1	0.48	-
ViTDet	43.0	50.9	64.3	32.0	0.47	12.1
STE-YOLO	72.8	74.5	79.0	50.0	0.72	61.40

4.8. Experimental Results

In Figure 7, the performance of our model SET-YOLO (on the right) on the NEU dataset is illustrated, showcasing the predicted bounding boxes, defect categories, and confidence scores. Compared to the baseline model YOLOv8 (on the left), our model successfully identifies a greater number of subtle and minor defects, highlighting its superior capability in defect detection.

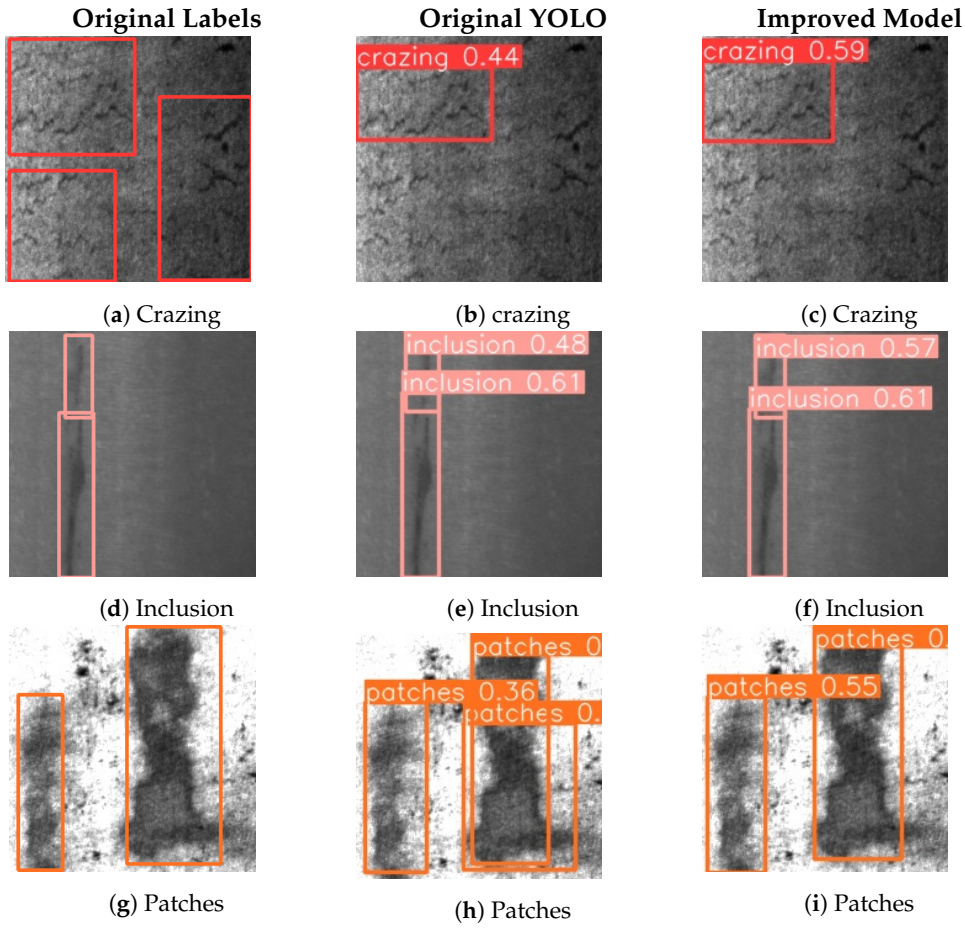
Figure 9 displays, from left to right, the original image, the detection results from the original YOLOv8, and the detection results from the improved STE-YOLO model, with each row dedicated to a specific type of surface defect. For crazing, the improved model shows a higher confidence score of 0.59 compared to the original model's 0.44. In the case of inclusion, both models detect similar defects with confidence scores up to 0.61. When identifying patches, the improved model performs more accurately with a score of 0.55, versus 0.36 for the original model, while also reducing incorrect bounding boxes. For pitted surface detection, the improved model achieves a higher confidence score of 0.80, compared to the original model's 0.60. Regarding the rolled-in scale, the improved model demonstrates more accurate bounding box localization. Lastly, for scratches, the improved model correctly labels the defects as "scratches", whereas the original model uses the label "inclusion scratches". This

comparison demonstrates the enhanced capability of the improved STE-YOLO model to accurately detect and categorize various surface defects on strip steel.

Furthermore, as shown in Table 4, while the overall performance of STE-YOLO is superior, certain defect types, such as crazing, exhibit relatively lower mAP scores. This suggests that the model faces challenges in detecting such defects due to their subtle features and similarities with the background.

When focusing on the crazing defect, we observe a trade-off between precision and recall. Specifically, while the model may achieve an acceptable recall, indicating that it identifies a significant portion of the actual crazing defects, it simultaneously struggles with precision, meaning it has a higher tendency to misclassify non-defect areas as crazing. This trade-off can be attributed to the inherent characteristics of crazing defects, which often blend in with the background and have irregular patterns. The model's attempts to capture even the subtlest crazing features could also lead to increased false positives, thereby lowering precision.

This observation indicates that while STE-YOLO achieves overall improvements in detection, there remains room for enhancement in handling certain types of defects, particularly those with low contrast and ill-defined boundaries. Future research directions could involve exploring more advanced feature extraction techniques, such as incorporating multi-scale information or developing more effective classification mechanisms to achieve a better balance between precision and recall for challenging defect types like crazing. In particular, attention mechanisms that focus on the shape and structure of these defects could improve performance.

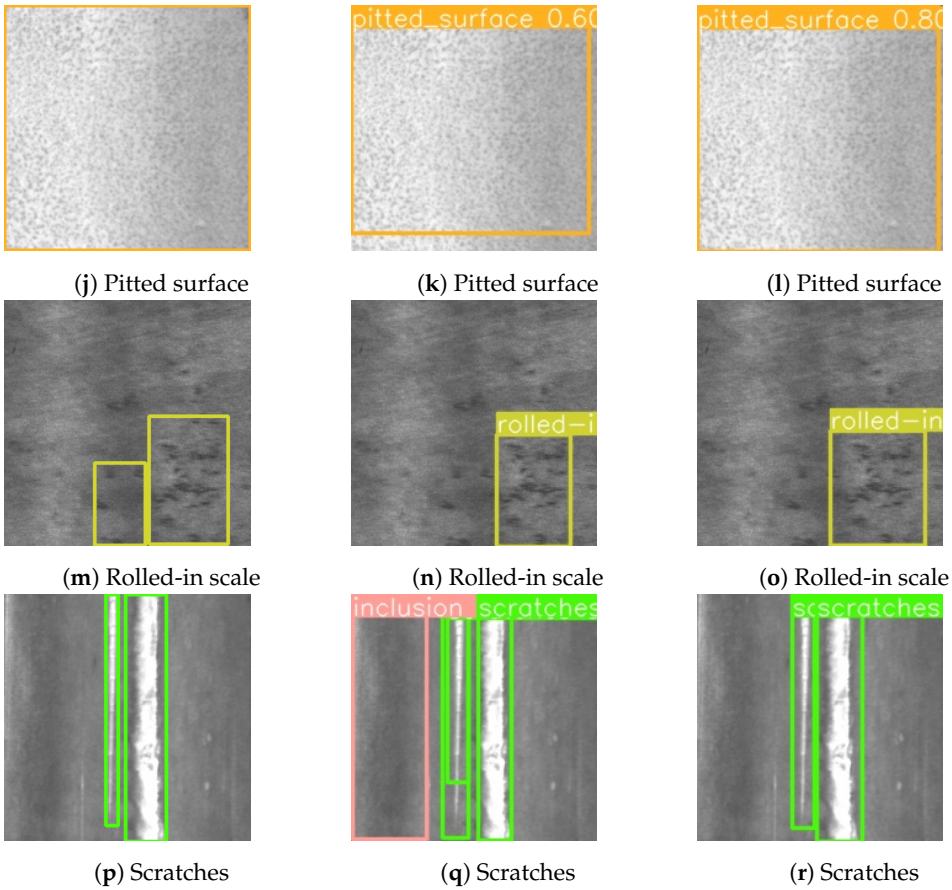


(**a**) Crazing (**b**) crazing (**c**) Crazing

(**d**) Inclusion (**e**) Inclusion (**f**) Inclusion

(**g**) Patches (**h**) Patches (**i**) Patches

Figure 9. *Cont.*



(**j**) Pitted surface (**k**) Pitted surface (**l**) Pitted surface

(**m**) Rolled-in scale (**n**) Rolled-in scale (**o**) Rolled-in scale

(**p**) Scratches (**q**) Scratches (**r**) Scratches

Figure 9. This figure presents a visual comparison of defect detection performance between the original YOLOv8 model and the improved STE-YOLO model on the NEU-DET dataset. In these figures, different colors are used to categorize various types of surface defects; Crazing is highlighted in red, inclusions are in pink, patches are in orange, pitted surface is in beige, rolled-in scale is in yellow, and scratches are in green.

5. Discussions

This study uses YOLOv8 as the defect detection model, leveraging its significant advantages in computational overhead and performance. Compared with traditional object detection algorithms, YOLOv8's superior processing speed and detection accuracy enable efficient data handling. Despite its excellent performance, YOLOv8 also has some limitations. Firstly, its ability to detect small objects is relatively weak, which may lead to missed detections. For example, when detecting object edges or small defects, the model may not reliably identify these features. Secondly, YOLOv8 may have difficulty accurately locating low-detection targets and is easily interfered with by background noise. These drawbacks significantly limit its application in high-precision requirements, especially in high-speed, high-interference industrial environments.

In our numerical experiments, the proposed model demonstrated strong performance in detecting defects such as patches and scratches, accurately identifying and classifying these defects. However, when defects are similar in color or texture to the background, the model's performance is unsatisfactory, which may be a drawback of this type of model. This issue may be due to the differences between the model design and the dataset characteristics, highlighting the difficulty of the algorithm in distinguishing subtle defects from their environment. Furthermore, the lower mAP scores for crazing defects indicate the model's weakness in identifying defects when the feature is similar to the background. In particular, according to Table 4, the mAP score of crazing defects is only 0.485, which is far lower than other types of defects. We speculate that the complex features of crazing defects

make the model vulnerable to background interference. In addition, we observed that the model struggles to achieve both high precision and recall for crazing defects, demonstrating the trade-off between these two metrics. Therefore, further research is required to enhance the model's capabilities in such situations.

6. Conclusions

In the production of steel strips, unforeseen defects often occur due to changes in production technology and external factors. The morphology of similar defects on the surface of steel strips varies, influenced by electromagnetic, light, and background noise, with different types of defects often having similarities. This mixed interference poses a significant challenge to the robustness of defect detection models. Our proposed research is based on an improved detection algorithm of YOLOv8, which meets the accuracy requirements for defect detection in steel strip production and the real-time speed requirements during production. The novelty of this algorithm lies in integrating the GhostConv module into the backbone network of YOLOv8, using the BoTNet structure to enhance the connection between the backbone and neck, and introducing the C2f_LSKA attention module. Compared with the original YOLOv8 model, the proposed algorithm shows significant improvements in detecting six typical defects in the NEU-DET dataset. However, the algorithm still has certain limitations in detecting Craze defects, performing relatively weakly.

Future work will focus on improving the susceptibility of STE-YOLO to background noise interference. One possible approach is to use shape information as auxiliary data to enable the model to locate objects more accurately. Alternatively, combining bounding box regression with shape constraints could improve STE-YOLO's ability to identify and locate low-contrast defects.

In summary, our research not only demonstrates the strengths and weaknesses of YOLOv8 in defect detection but also suggests potential areas for future improvement. These improvements provide a clear direction for future research. Through continuous improvement and iteration, we hope to enhance the applicability of our model, making it more effective in the task of detecting defects in steel strips.

Author Contributions: Conceptualization, D.L. and E.W.; Methodology, E.W., L.Z. and D.L.; Software, E.W., Z.L. and L.Z.; Validation, E.W., C.Z. and D.L.; Formal Analysis, E.W. and Z.L.; Investigation, E.W.; Resources, E.W.; Data Curation, E.W. and C.Z.; Writing—Original Draft Preparation, L.Z. and C.Z.; Writing—Review & Editing, C.Z. and Y.Y.; Visualization, E.W., L.Z. and D.L.; Supervision, D.L. and C.Z.; Project Administration, E.W.; Funding Acquisition, D.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the NSFC, grant number 61806024; Jilin Province Science and Technology Development Plan Key Research and Development Project, grant number 20210204050YY; Wuxi University Research Start-up Fund for Introduced Talents, grant numbers 2023r004 and 2023r006; Wuxi "Light of Taihu Lake" Technology Research Project, grant number K20241044.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All the data mentioned in this paper are available through the corresponding author.

Acknowledgments: We thank Tian Zhang for providing additional technical guidance and support for this paper.

Conflicts of Interest: The authors declare that they have no conflicts of interest regarding the publication of this paper.

References

1. Božič, J.; Tabernik, D.; Skočaj, D. Mixed supervision for surface-defect detection: From weakly to fully supervised learning. *Comput. Ind.* **2021**, *129*, 103459. [CrossRef]
2. Kim, S.; Kim, W.; Noh, Y.K.; Park, F.C. Transfer learning for automated optical inspection. In Proceedings of the 2017 International Joint Conference on Neural Networks (IJCNN), Anchorage, AK, USA, 14–19 May 2017; pp. 2517–2524.
3. Forsyth, D.A.; Ponce, J. *Computer Vision: A Modern Approach*; prentice hall professional technical reference; Savvas Learning Company: Hoboken, NJ, USA, 2002.
4. Kou, X.; Liu, S.; Cheng, K.; Qian, Y. Development of a YOLO-V3-based model for detecting defects on steel strip surface. *Measurement* **2021**, *182*, 109454. [CrossRef]
5. Song, K.; Yan, Y. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Appl. Surf. Sci.* **2013**, *285*, 858–864. [CrossRef]
6. Shumin, D.; Zhoufeng, L.; Chunlei, L. AdaBoost learning for fabric defect detection based on HOG and SVM. In Proceedings of the 2011 International Conference on Multimedia Technology, Hangzhou, China, 26–28 July 2011; pp. 2903–2906.
7. Chondronasios, A.; Popov, I.; Jordanov, I. Feature selection for surface defect classification of extruded aluminum profiles. *Int. J. Adv. Manuf. Technol.* **2016**, *83*, 33–41. [CrossRef]
8. Gibert, X.; Patel, V.M.; Chellappa, R. Deep multitask learning for railway track inspection. *IEEE Trans. Intell. Transp. Syst.* **2016**, *18*, 153–164. [CrossRef]
9. Tao, X.; Xu, D.; Zhang, Z.T.; Zhang, F.; Liu, X.L.; Zhang, D.P. Weak scratch detection and defect classification methods for a large-aperture optical element. *Opt. Commun.* **2017**, *387*, 390–400. [CrossRef]
10. Girshick, R.; Donahue, J.; Darrell, T.; Malik, J. Rich feature hierarchies for accurate object detection and semantic segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 23–28 June 2014; pp. 580–587.
11. Girshick, R. Fast r-cnn. *arXiv* **2015**, arXiv:1504.08083.
12. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2016**, *39*, 1137–1149. [CrossRef]
13. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C.Y.; Berg, A.C. Ssd: Single shot multibox detector. In Proceedings of the Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016; Proceedings, Part I 14; Springer: Berlin/Heidelberg, Germany, 2016; pp. 21–37.
14. Redmon, J. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 26 June–1 July 2016.
15. Redmon, J.; Farhadi, A. YOLO9000: Better, faster, stronger. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 21–26 July 2017; pp. 7263–7271.
16. Du, Y.; Chen, H.; Fu, Y.; Zhu, J.; Zeng, H. AFF-Net: A Strip Steel Surface Defect Detection Network via Adaptive Focusing Features. *IEEE Trans. Instrum. Meas.* **2024**, *73*, 2518514. [CrossRef]
17. Yi, L.; Li, G.; Jiang, M. An end-to-end steel strip surface defects recognition system based on convolutional neural networks. *Steel Res. Int.* **2017**, *88*, 1600068. [CrossRef]
18. Li, M.; Wang, H.; Wan, Z. Surface defect detection of steel strips based on improved YOLOv4. *Comput. Electr. Eng.* **2022**, *102*, 108208. [CrossRef]
19. Feng, X.; Gao, X.; Luo, L. X-SDD: A New Benchmark for Hot Rolled Steel Strip Surface Defects Detection. *Symmetry* **2021**, *13*, 706. [CrossRef]
20. Dong, X.; Li, Y.; Fu, L.; Liu, J. Edge-aware interactive refinement network for strip steel surface defects detection. *Meas. Sci. Technol.* **2024**, *36*, 016222. [CrossRef]
21. Terven, J.; Córdova-Esparza, D.M.; Romero-González, J.A. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Mach. Learn. Knowl. Extr.* **2023**, *5*, 1680–1716. [CrossRef]
22. Tulbure, A.A.; Tulbure, A.A.; Dulf, E.H. A review on modern defect detection models using DCNNs–Deep convolutional neural networks. *J. Adv. Res.* **2022**, *35*, 33–48. [CrossRef]
23. Howard, A.G. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv* **2017**, arXiv:1704.04861.
24. Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.C. Mobilenetv2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 4510–4520.
25. Howard, A.; Sandler, M.; Chu, G.; Chen, L.C.; Chen, B.; Tan, M.; Wang, W.; Zhu, Y.; Pang, R.; Vasudevan, V.; et al. Searching for mobilenetv3. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 1314–1324.
26. Barile, C.; Casavola, C.; Pappalettera, G.; Kannan, V.P. Damage Progress Classification in AlSi10Mg SLM Specimens by Convolutional Neural Network and k-Fold Cross Validation. *Materials* **2022**, *15*, 4428. [CrossRef]

27. Liu, Z.; Li, D.; Zhu, Q.; Zhang, C.; Ma, J.; Zhao, J. Intelligent method to experimentally identify the fracture mechanism of red sandstone. *Int. J. Miner. Metall. Mater.* **2023**, *30*, 2134–2146. [CrossRef]
28. Zhang, X.; Zhou, X.; Lin, M.; Sun, J. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–22 June 2018; pp. 6848–6856.
29. Ma, N.; Zhang, X.; Zheng, H.T.; Sun, J. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 116–131.
30. Cao, J.; Bao, W.; Shang, H.; Yuan, M.; Cheng, Q. GCL-YOLO: A GhostConv-Based Lightweight YOLO Network for UAV Small Object Detection. *Remote Sens.* **2023**, *15*, 4932. [CrossRef]
31. Firdiantika, I.M.; Lee, S.; Bhattacharyya, C.; Jang, Y.; Kim, S. EGCY-Net: An ELAN and GhostConv-Based YOLO Network for Stacked Packages in Logistic Systems. *Appl. Sci.* **2024**, *14*, 2763. [CrossRef]
32. Zhang, J.; Wang, H.; Tian, Y.; Liu, K. An accurate fuzzy measure-based detection method for various types of defects on strip steel surfaces. *Comput. Ind.* **2020**, *122*, 103231. [CrossRef]
33. Yue, X.; Ma, G.; Liu, F.; Gao, X. Research on image classification method of strip steel surface defects based on improved Bat algorithm optimized BP neural network. *J. Intell. Fuzzy Syst.* **2021**, *41*, 1509–1521. [CrossRef]
34. Ashour, M.W.; Khalid, F.; Abdul Halin, A.; Abdullah, L.N.; Darwish, S.H. Surface defects classification of hot-rolled steel strips using multi-directional shearlet features. *Arab. J. Sci. Eng.* **2019**, *44*, 2925–2932. [CrossRef]
35. Guan, S.; Lei, M.; Lu, H. A Steel Surface Defect Recognition Algorithm Based on Improved Deep Learning Network Model Using Feature Visualization and Quality Evaluation. *IEEE Access* **2020**, *8*, 49885–49895. [CrossRef]
36. Deshpande, A.M.; Minai, A.A.; Kumar, M. One-Shot Recognition of Manufacturing Defects in Steel Surfaces. *Procedia Manuf.* **2020**, *48*, 1064–1071. [CrossRef]
37. Zou, Y.; Fan, Y. An Infrared Image Defect Detection Method for Steel Based on Regularized YOLO. *Sensors* **2024**, *24*, 1674. [CrossRef]
38. Guo, Z.; Wang, C.; Yang, G.; Huang, Z.; Li, G. MSFT-YOLO: Improved YOLOv5 Based on Transformer for Detecting Defects of Steel Surface. *Sensors* **2022**, *22*, 3467. [CrossRef]
39. Xia, K.; Lv, Z.; Zhou, C.; Gu, G.; Zhao, Z.; Liu, K.; Li, Z. Mixed Receptive Fields Augmented YOLO with Multi-Path Spatial Pyramid Pooling for Steel Surface Defect Detection. *Sensors* **2023**, *23*, 5114. [CrossRef]
40. Li, S.; Wu, C.; Xiong, N. Hybrid Architecture Based on CNN and Transformer for Strip Steel Surface Defect Classification. *Electronics* **2022**, *11*, 1200. [CrossRef]
41. Wang, G.; Chen, Y.; An, P.; Hong, H.; Hu, J.; Huang, T. UAV-YOLOv8: A Small-Object-Detection Model Based on Improved YOLOv8 for UAV Aerial Photography Scenarios. *Sensors* **2023**, *23*, 7190. [CrossRef]
42. Lou, H.; Duan, X.; Guo, J.; Liu, H.; Gu, J.; Bi, L.; Chen, H. DC-YOLOv8: Small-Size Object Detection Algorithm Based on Camera Sensor. *Electronics* **2023**, *12*, 2323. [CrossRef]
43. Jocher, G.; Chaurasia, A.; Qiu, J.; Stoken, A. YOLOv8: Real-Time Object Detection. 2023. Available online: https://github.com/ultralytics/ultralytics (accessed on 23 December 2024).
44. He, Y.; Song, K.; Meng, Q.; Yan, Y. An End-to-End Steel Surface Defect Detection Approach via Fusing Multiple Hierarchical Features. *IEEE Trans. Instrum. Meas.* **2020**, *69*, 1493–1504. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.